KillerPDF

Technical Specification

& Complete Feature Brochure

VERSION 1.7.1

A native Windows PDF viewer, annotator and light editor.
Single self-contained executable. Built on WPF and PDFium.
This document is itself a KillerPDF showcase - open it in the app
and explore the outline, the forms, and the annotation pages.

About the Maker

KillerPDF

Steve the Killer

Field technician - systems administrator - builder of Killer Tools

KillerPDF was born out of a ticket queue.

I spend my days as a field tech and systems administrator at a large MSP, and a steady share of that work is always the same shape: a user can't fill a PDF, a form won't flatten, a license won't activate, a viewer throws an error on a file that should just *open*. Adobe Acrobat tickets - install it, repair it, license it, work around it - added up to hours every week that never felt like they were actually fixing anything.

So I built the tool I wished my users had: one fast, self-contained executable that opens the broken files, fills forms, signs documents, OCRs scans and gets out of the way. No installer, no account, no subscription. It is deliberately overbuilt for the encrypted, broken-xref and rotated-and-clipped PDFs that make other viewers give up - because those are the files that generate tickets. Everything in this brochure is real and shipping.

> "I got tired of closing the same Adobe ticket. So I wrote a PDF app that just works, ships as one .exe, and never asks the user to log in. That app is KillerPDF."

KillerPDF is free software (GPL-3.0). Find it, and everything else I build, at **KillerPDF.net**.

About this Brochure

This document is the complete technical specification and feature list for **KillerPDF version 1.7.1**, and a working showcase of the app itself.

Open it inside KillerPDF and it becomes a tour: the outline is a deep nested bookmark tree (now editable in the sidebar), Part IV has a real interactive AcroForm and a printed-style form to mark up, and the annotation pages are seeded for screenshots. Every tool, theme, shortcut and constant here is taken directly from the source.

How this document is organized

- **Part I** - the platform: tech stack, architecture, render pipelines, view modes and the calculation deep-dive.
- **Part II** - every tool, in the GUI and under the hood, including the command-line interface.
- **Part III** - reference: keyboard map, themes, languages, robustness, standards conformance, the hard numbers, and a glossary.
- **Part IV** - a live showcase you can interact with in the viewer.

Any page here makes a clean example document for marketing captures.

Table of Contents

Platform & Architecture

- The technology and ideas the app is built on
- Single-class WPF architecture, two render pipelines
- A zoom-stable coordinate space shared by every view

Product & Tech Stack

KillerPDF is a native Windows PDF viewer, annotator and light editor shipped as a single self-contained executable. It renders with PDFium (Docnet.Core), reads text with PdfPig and writes the PDF object model with PdfSharpCore, vendored into the repo and patched for standards conformance. The WPF shell remains in **MainWindow**, while each document pane is a reusable **PdfViewer** control with its own view state, tabs and render surfaces. The app spans about 42,000 lines of C# across 120 source files.

Product & build

Attribute	Value
Version	1.7.1 (mirrors the csproj <Version>)
Target framework	.NET Framework 4.8 (net48), WinExe, x64
UI toolkit	WPF, custom chrome with native resize and snap
Distribution	Single .exe, portable by default; an optional install (per-user, or all-users via an admin prompt, or /silent for unattended deployment) adds Program Files and Start Menu entries. Costura embeds DLLs, native OCR libs self-extract
Render engine	PDFium via Docnet.Core 2.6
Text engine	PdfPig 0.1.14 - search, selection, font sniffing
Write engine	PdfSharpCore 1.3.67, vendored under third_party/ with six conformance patches
Signing engine	PDFsharp 6.2.4 (separate namespace) - CMS / SHA-256
Image codecs	SixLabors.ImageSharp 2.1.13 - import, clipboard paste, signatures
OCR	Tesseract 5.2 - native libs embedded, packs on demand (10 languages)
Command line	Full headless CLI: merge, split, decrypt, rasterize, flatten, print, OCR, batch-resave

At a glance

- Four view modes sharing one coordinate space; a full annotation suite (text, highlight, ink, lines, shapes, images, signatures, covers).
- Editing: crop, rotate, scale, flip, deskew, perspective correction, page reorder / insert / delete, bookmark editing, stamp, watermark, page numbering.
- Interactive AcroForm filling, link handling, full-text search, jump history, OCR and cryptographic signing.
- Multi-document tabs with per-tab sessions and an LRU cache; split pane (F10) shows two documents side by side in one window; six themes, ten UI languages.
- Themed Open and Save dialogs - places rail, folder tree, list / icon / details views - replace the stock Windows ones for every file operation: opening, saving, merging, extracting pages, flatten, image export/import, signature and certificate picking, OCR output and zip export.

Architecture Overview

One shell, reusable document views. MainWindow owns the shared toolbar, sidebar, dialogs and application-level workflows. Each pane is a **PdfViewer** control that owns its view state, tab strip, render surfaces and document interaction code. Two control instances create split pane without a second window. Rendering, annotations, forms, links, crop, selection, text editing, tabs and zoom are divided across 16 `Controls/Viewer/PdfViewer.*` partials. Bridge files keep the remaining shell features connected while the refactor continues; cross-cutting work lives under `Services/` and feature controllers.

Two render pipelines

Single, Two-Page and Grid render into the primary tile (`PageImage` + `_annotationCanvas`) through `RenderPage(idx)`, with extra tiles built by `RenderAdditionalPages`. Continuous mode owns a separate path: `SetupContinuousView` builds one overlay per page and `RenderContinuousPages` streams bitmaps on a background thread. The continuous strip is virtualized - only a window of pages around the viewport holds real bitmaps, so a 243-page image-heavy PDF costs a few hundred MB instead of gigabytes.

```
INVARIANT RenderPage is Single / Grid / TwoPage only and no-ops in Continuous - it calls
ClearSecondaryPages, which would otherwise repoint every overlay at the hidden primary
tile and paint annotations off-screen. (The coordinate math is in the deep-dive.)
```

Per-tab sessions & the LRU cache

Each open document is a `DocumentSession` holding its zoom, fit and view mode, grid column count, tool, page, scroll offsets and the per-document dictionaries (annotations, render dims, rotations, form values, undo and redo stacks, jump history). Switching tabs re-points the live fields at the session by reference. Each session also carries a frozen-bitmap `RenderCache` keyed by (page, size-bucket, rotation), so returning to a recent tab skips PDFium entirely; whole-tab caches evict beyond the three most recent. Fit, zoom, view and page persist per file path across restarts (capped at 40 documents); lazy tabs defer loading until first activated.

View Modes

KillerPDF offers four view modes, switchable from the toolbar or with F5-F8. All four place annotations against the same 2048-longest-side coordinate space, so marks never drift when you change how pages are laid out.

Mode	Behavior
Continuous (F5)	Vertical scroll of every page; overlays built up front, bitmaps streamed asynchronously and released once far off-screen. The default mode. The current page follows the viewport; clicks never yank the view.
Single (F6)	One page in the primary tile; fit and zoom math use the selected page's render dimensions.
Two-Page (F7)	Primary tile plus one secondary tile side by side, paired as spreads (0,1)(2,3)... Arrow keys and PgUp / PgDn move one spread at a time.
Grid (F8)	A thumbnail overview of the whole document; the column count is the single source of truth for grid zoom and is stored per tab. The status bar counter follows the tile nearest the viewport center.

Single, Two-Page and Grid share the primary panel; Continuous uses its own stacked panel. Secondary tiles key the render cache by their rounded primary-tile width, so zooming reuses cached bitmaps. A moon toggle at the foot of the sidebar (or the N key) inverts the document colors for dark-mode reading - display only, so saving, printing, export and OCR keep the true colors.

Independently of view mode, F10 splits the window itself into two panes side by side, each running its own tabs and its own choice of the four view modes above - see Split Pane in Part II for the full behavior.

How It's Calculated: Rendering & Zoom

Two numbers describe every page: where things *are* (zoom-independent) and how many pixels we *draw* (zoom-dependent). Keeping them separate is what makes annotations stay put while text stays crisp.

1. The render-dim space

Every page is normalized so its longest side is exactly 2048 units; annotation coordinates are stored here, identical at every zoom and across all four pipelines.

```
maxDim = max(pageWidthPt, pageHeightPt)
rdW = round(2048 * pageWidthPt / maxDim)
rdH = round(2048 * pageHeightPt / maxDim) // longest side -> 2048
```

2. Decoupled bitmap resolution

The bitmap rasterizes at a resolution that follows DPI and zoom, so a 3x page is drawn from 3x the pixels rather than upscaled. The 6144 px ceiling bounds memory.

```
scaledMax = min( 6144, int( 2048 * max(dpiScaleX, dpiScaleY) * max(1.0, zoom) ) )
```

3. The coordinate Y-flip

PDF uses a bottom-left origin in points; WPF canvases a top-left origin in DIP. Every mark crosses that boundary:

```
sx = renderW / pdfW sy = renderH / pdfH
canvasX = left * sx
canvasY = renderH - top * sy // flip the vertical axis
```

rdW/rdH are rounded once and reused, so the same integers drive the primary, continuous and grid tiles - a mark in one mode lands on the same pixel in every other.

How It's Calculated: Interaction

The same precision applies to what you do with the mouse.

Cursor-anchored zoom

Ctrl+wheel zooms around the cursor by a constant 10% ratio per notch: the view scales instantly while the wheel is moving and the crisp high-resolution re-render happens once when the wheel rests. The cursor position and scroll offsets are captured *before* the zoom changes, then the offsets that keep that point fixed are applied once layout settles.

```
ratio = newZoom / oldZoom
newHOff = (oldHOff + cursorX) * ratio - cursorX
newVOff = (oldVOff + cursorY) * ratio - cursorY // clamped at >= 0
```

Grid zoom by column count

Grid snaps to a whole number of columns: the count is authoritative and the zoom is derived from it, so the grid lays out exactly N pages with no leftover gap.

```
GridZoomForN(n) = (viewportW - 24) / ( n * (rdW + 12) )
```

Cache keys & eviction

The render cache is keyed by the tuple that uniquely determines a page's pixels, so a hit is always safe to reuse; whole-tab caches drop off the end of a three-deep most-recently-used list.

```
key = ( pageIndex, sizeBucket, rotation )
sizeBucket = scaledMax (primary) or round(primaryDipW) (tiles)
evict : keep the 3 most-recently-used tabs; clear the rest
```

II

Tools & Features

- Every annotation, editing and document tool
- How each behaves in the GUI
- What happens under the hood, with real names

KillerPDF

Text & Typewriter

Text [T / 1]

Drop a text box anywhere, or re-edit existing PDF text in place.

IN THE GUI

Click to place a box and type (Enter saves, Escape cancels); double-click existing text to re-edit. A floating bar sets color, opacity, fill, point size, font and Bold / Italic / Strike / Underline.

UNDER THE HOOD

> PlaceTextBox builds a flat WPF TextBox with live resize handles; CommitActiveTextBox converts it to a TextAnnotation. Re-editing drops a page-color CoverAnnotation (paired by PairId) over the original; font and size are sniffed from PdfPig with a mojibake guard, falling back to Segoe UI 14pt.

Highlight, Strikethrough & Underline

Highlight / Strikethrough / Underline [H / 2]

Three text-flowing mark styles sharing one annotation type, with an optional eraser.

IN THE GUI

Drag across words and the markup hugs each line, browser-style, instead of laying down one rectangle - across lines, paragraphs and (in continuous view) across pages. One gesture makes one grouped annotation per page that selects, moves, deletes and undoes as a unit. On pages with no text layer the tools show a status hint instead of drawing a box; the highlight eraser keeps its rectangle.

UNDER THE HOOD

```
The Select tool's drag tracks the actual character runs in reading order
(Services/TextRunService.cs, PdfPig + PDFium char boxes). CommitFlowingHighlight turns the
per-line rects into HighlightAnnotations grouped by GroupId, committed as one undo step. Each is
tagged with a HighlightStyle and RGBA color.
```

Ink (Draw) & Line

Draw & Line `[D / 5 - L / 3]`

Freehand ink and straight lines, both backed by one ink annotation type.

IN THE GUI

Draw paints freehand strokes and keeps drawing without re-arming; Line drags a straight segment with an optional Level snap. The bar offers color, opacity, width and an eraser (Draw) or level (Line).

UNDER THE HOOD

> Both create an InkAnnotation (points + stroke width + RGBA, default red 2.0px). Draw accumulates points on move; Line keeps two points. A minimum-length guard rejects click jitter.

Shapes

Shapes [4]

Rectangle, ellipse and free-form polygon markers, each with an optional fill.

IN THE GUI

Box keeps the classic drag-a-filled-rectangle gesture the highlighter used to have; ellipse and polygon are closed outlines that move, resize, flatten and print like any other drawing. Free-form places points click by click - click the first point (its target lights up as you approach) or double-click to close, Esc cancels, Backspace removes the last point. The tool shares the draw bar's color, size and opacity, with a mini-shape sub-mode picker and a Fill toggle.

UNDER THE HOOD

```
New Shapes.cs partial (EditTool.Shape + ShapeKind). Box+fill is a HighlightAnnotation; box no-fill and ellipse and polygon ride the InkAnnotation pipeline with a closed flag and optional FillR/G/B/A, rendering as a WPF Polygon and exporting via gfx.DrawPolygon (fill then stroke).
```

Image & Cover

Image `[I / 6]`

Place a picture on the page; covers provide whiteout for text replacement.

IN THE GUI

Pick Image, click and choose a file (or just paste with Ctrl+V); it lands at 50% scale, corner-resizable. Covers are the opaque whiteout half of an in-place text re-edit.

UNDER THE HOOD

```
ImageAnnotation stores Base64 bytes with the decoded bitmap cached, drawn at SourceWidth*Scale.
Decoding runs through SixLabors.ImageSharp 2.1.13, and images with broken DPI metadata (WhatsApp
photos, some scans) are kept within Adobe's supported 3-14,400 point page range. CoverAnnotation
derives from HighlightAnnotation with forced-opaque alpha, paired by PairId and flattened solid on
export.
```

Crop

Crop [C / 8]

Trim pages with numeric precision, on one page or all.

IN THE GUI

A default box inset 8% from the edges plus a bar with X / Y / W / H inputs, a unit selector (points, inches or %), current- or all-pages scope, Remove-crop and Apply / Cancel.

UNDER THE HOOD

> `ApplyCrop converts the canvas rect to PDF coordinates (rotation-aware via CanvasToPdfRect), clamps to the media box, writes /CropBox and /TrimBox with PdfSharpCore, then saves and reloads. Every save also strips degenerate zero-size crop boxes, healing files damaged by older builds.`

Links

Links

Existing PDF links become clickable; you can copy or remove them.

IN THE GUI

Links become hand-cursor overlays: left-click follows an internal jump or external URI; right-click offers Copy URL or Remove Link. Internal jumps are recorded in the jump history, so Alt+Left retraces them. An optional Confirm before opening links setting, on the About card beside the recent-files toggle, asks before a link opens in your browser; it is off by default so links stay immediate unless you want the check.

UNDER THE HOOD

```
RenderPageLinks reads each page's /Annots, parses /Link /Rect and the /A action and builds transparent overlays; object-stream PDFs fall back to PDFium link enumeration through a cached handle that is released before every save. On save, StripLinkAnnotationBorders removes borders so they don't strobe.
```

Forms (AcroForm)

Forms

Fill interactive text fields, checkboxes, radios and dropdowns.

IN THE GUI

Widget fields overlay the page with a cream background and accent-on-focus border; text fields show a font-size stepper as an inline flyout that follows the field through scroll and zoom. Checkboxes, radios and dropdowns are editable and written back on save.

UNDER THE HOOD

```
Widgets parse into FormFieldInfo (type /Tx, /Btn or /Ch; /Rect with full 0/90/180/270 rotation handling; /DA font size). Values persist by object number / field name and are written by WriteFormValuesToDocument, which regenerates each field's appearance and sets /NeedAppearances so other readers redraw correctly. Try the Interactive Feedback Form in Part IV.
```

Stamp, Watermark & Page Numbers

Stamp / Watermark `[S / 0]`

Two independent overlays - page numbers and a text or image watermark.

IN THE GUI

A live-preview editor with a page stepper. Page numbers take a start value, a format ({n} current, {N} total), size, color, a range like 1-3,5, eight positions plus custom and a spread mirror. Watermarks take text or an image, size / color or scale, opacity, angle and position. Stamps apply to a clean document with no other markup, and clear again by applying with both sections off.

UNDER THE HOOD

```
A StampSpec holds every setting (watermark default DRAFT 64pt 25% at 45 degrees).
DrawStampsOnDocument flattens via PdfSharpCore XGraphics - watermark under, numbers on top, image
opacity baked through a color matrix, rotation via translate-plus-rotate.
```

Search

Search `[Ctrl+F - F3]`

Debounced full-text search with persistent per-page highlights.

IN THE GUI

Ctrl+F toggles a draggable search bar (250ms debounce); the status shows current / total, and F3 / Shift+F3 step to the next / previous match from anywhere (F3 opens search when it isn't showing). Enter, Shift+Enter and the wheel walk the matches too.

UNDER THE HOOD

```
SearchService.Search scans each page's PdfPig GetWords() case-insensitively (single- and
multi-word) for per-page rectangles. ApplySearchHighlights repaints them on every render - the
current match a brighter accent outline, the rest dim.
```

OCR

OCR [Ctrl+Shift+O - Ctrl+Shift+I]

On-device text recognition in ten languages, with a searchable-PDF text layer.

IN THE GUI

Ctrl+Shift+O OCRs the page to the clipboard; Ctrl+Shift+I drags a region; a language menu manages models and Escape cancels. You can also write the recognized text back as an invisible, transparent layer over the image, making a scan fully selectable and searchable without changing how it looks.

UNDER THE HOOD

> `OcrNativeBootstrap self-extracts the embedded x64 Tesseract / Leptonica libraries to a per-version cache. Ten languages (matching the interface locales) download on demand in two tiers: a standard model (~4 MB) and a higher-accuracy model (~14 MB). Pages render at 2600px (~300 DPI) and feed OcrService.RecognizeBgra, returning text and a mean confidence.`

Digital & Visual Signing

Digital Signing (cryptographic)

Certificate-based CMS signatures with whole-chain SHA-256.

IN THE GUI

Pick a certificate from a .pfx / .p12 file or the Windows store, add an optional reason, location and contact, and write a signed copy.

UNDER THE HOOD

> Built on PDFsharp 6.2. WindowsCertificateStore enumerates the personal store; KillerCmsSigner implements IDigitalSigner on .NET SignedCms (CNG-capable, so token / cloud keys can prompt for a PIN), with SHA-256, the whole chain and a 16,384-byte /Contents reservation. Because any later edit breaks a signature's digest, resaving a signed file strips the dead signature value rather than shipping one that fails strict validation.

Visual Signatures `[G / 7]`

Hand-drawn or imported signatures, saved for reuse.

IN THE GUI

A draw pane records strokes at three pen widths; saved signatures and initials live in a draggable popup that remembers its position and drop straight into an AcroForm field.

UNDER THE HOOD

> SignatureStore persists them as JSON in %LOCALAPPDATA%. Placement creates a SignatureAnnotation (scale 0.5 full, 0.3 initials) that can bind to a form field.

Print

Print [Ctrl+P]

A real print-preview with N-up, scaling, margins, alignment and manual duplex.

IN THE GUI

Preview with N-up (1 / 2 / 4 / 6 / 9), per-page scale (fit, 100% or custom), alignment, margins, orientation, page navigation and a rendering progress label. A Pages selector prints All, Odd only or Even only on top of the page range - print the odds, flip the stack, print the evens for manual duplex. Copies and custom scale are numeric fields with spinners; the dialog remembers the last printer, orientation, color and two-sided choice.

UNDER THE HOOD

```
Built on System.Printing and ReachFramework. On Print, the selected pages re-rasterize at 300 DPI
from a flattened copy (annotations baked in), compose into a FixedDocument and go to the spooler
via PrintDialog.
```

Rotate & Transform

Rotate / Transform [R / 9]

Quick 90-degree turns or a full transform with scale, flip, straighten and perspective correction.

IN THE GUI

Quick rotations are on the context menu and the R key; the Transform window adds angle, scale, flip, straighten-by-line, perspective correction and fixed- versus resize-page modes at full resolution, with a live preview. Rotate, Scale, Flip, Skew and Perspective sit in collapsible sections so the sidebar stays compact. Annotations on the page follow the transform.

UNDER THE HOOD

```
Structural rotation routes through SaveTempAndReload, which zeroes /Rotate before saving (PDFium
sizes from the unrotated MediaBox, so rotated content would clip) and restores it in memory.
ApplyPageTransform burns annotations in, composes perspective / flip / scale / rotate-with-expand
and re-encodes in one undoable operation.
```

Trapezoidal Perspective Correction

Perspective correction [R / 9]

Straighten photographed pages that taper or lean away from the camera.

IN THE GUI

Open Transform, expand Perspective and turn on Correct trapezoidal distortion. Four green handles appear at the page corners. Drag them onto the photographed page outline in top-left, top-right, bottom-right and bottom-left order, then Apply. The selected quadrilateral becomes a straight rectangular page and the active view redraws immediately.

UNDER THE HOOD

```
PerspectiveCorners stores four normalized points. PerspectiveWarp validates a convex, non-crossing quadrilateral, builds a square-to-quad projective homography and samples the source with bilinear interpolation at full transform resolution. The warp composes with rotation, deskew, scale and flip before SaveTempAndReload replaces the page; Ctrl+Z restores the original.
```

Best results

Place each handle on the real paper edge, not on the photo boundary. Keep the corner order intact; crossed, collapsed and concave selections are rejected before the document is changed.

```
input : photographed page quadrilateral
map : normalized rectangle -> selected source corners
sample : bilinear BGRA interpolation
output : straight rectangular page, immediately re-rendered
```

Page Operations & Import

Page operations

Reorder, insert, delete, duplicate, extract and split pages.

IN THE GUI

Rotate, delete, insert or append blank A4 pages, move up / down, duplicate, extract or split; dragging a sidebar thumbnail reorders pages.

UNDER THE HOOD

```
All structural edits route through SaveTempAndReload(keepAnnotations:true), which rebuilds the rotation and render-dim maps and refreshes thumbnails on a background thread.
```

Export & import

Render pages to images, and open images, folders and archives as pages.

IN THE GUI

Export pages as images renders to PNG or JPEG at a chosen DPI (24-1200, default 150) with an optional page range, burning in pending annotations and stamps. Open images (JPG, PNG, BMP, GIF, TIFF) as pages; folders recurse and .zip extracts to temp, with a choice to merge into one PDF or open each in its own tab.

UNDER THE HOOD

```
Export shares the CLI's --to-image pipeline and composites over white (a --transparent flag keeps raw PNG alpha). Import uses each image's own DPI (96 fallback); a 50-file cap (MaxDropFiles) prompts and truncates.
```

Split Pane

Split Pane [F10]

View two documents side by side in one window.

IN THE GUI

F10 splits the window into two panes, each a card of its own; the focused pane carries an accent ring, and clicking either pane moves the toolbar, sidebar and page list to act on it. A handle on each side of the divider resizes its own pane, and neither can be squeezed below a readable width from either direction. Drag a tab from one pane's strip to the other to move it across. F10 again closes the split; splitting a maximized or snapped window divides it evenly instead of squeezing the second pane down to its minimum.

UNDER THE HOOD

> The document view is now a self-contained control rather than part of the main window - about 8,700 lines moved out, every function verbatim and verified line by line - so a second instance of it can be hosted side by side with the first. The tab strip, toolbar and sidebar stay shared window-level state that simply acts on whichever pane last claimed focus.

Tabs, Outline & Thumbnails

Tabs `[Ctrl+Tab - Ctrl+W - Ctrl+Shift+W]`

Multi-document tabs with flicker-free, reconcile-in-place rendering.

IN THE GUI

Open many documents as tabs; the active tab stays visible with an overflow menu. Ctrl+Shift+W (or Close Other Tabs on a tab's right-click menu) closes every tab except the current one. Collapsing the sidebar is a smooth quarter-second slide.

UNDER THE HOOD

```
Backed by DocumentSession. The strip is reconciled in place via a session-to-element cache - no
re-parenting, so no flicker on switch, drag or resize. Lazy tabs defer loading until activated.
```

Outline & Thumbnails `[Ctrl+B]`

A dockable sidebar with an editable bookmark tree and lazy thumbnails.

IN THE GUI

A dockable sidebar carries a Pages (thumbnail) tab and an Outline tab that mirrors the PDF's bookmark tree (this document has a deep one, try it here) and edits it in place: add via the row at the top, rename inline (F2), nest, reorder, retarget and delete, with Ctrl / Shift multi-select and full undo.

UNDER THE HOOD

```
LoadOutlines walks _doc.Outlines into a TreeView; titles from password-protected files are
re-decoded to dodge mojibake, and bookmarks that point at named destinations resolve through the
catalog name-tree. Thumbnails lazy-load on a concurrency-limited thread at ~288px.
```

Redo & Jump History

Redo / Jump history `[Ctrl+Y - Alt+Left / Alt+Right]`

Browser-style history for both your edits and your movements through the document.

IN THE GUI

Ctrl+Y (or Ctrl+Shift+Z) re-applies undone actions: annotations, text edits, stamps, clears and document-level operations alike. Alt+Left / Alt+Right, or the mouse back / forward buttons, retrace bookmark, link, jump-box and Home / End jumps exactly like a browser.

UNDER THE HOOD

```
Redo entries mirror the _undoStack records; any new edit clears the redo chain, and both stacks
live per tab in the DocumentSession. Jump history is a pair of page stacks (back / forward) fed by
RecordNavJump at every hard jump; retracing pushes the current page onto the opposite stack.
```

Zoom & Undo

Zoom `[Ctrl + - Ctrl - - Ctrl+0 - Ctrl+1/2/3]`

Cursor-anchored zoom from 5% to 500%.

IN THE GUI

Ctrl+wheel zooms around the cursor; in Grid the wheel steps the column count. An editable zoom box offers fit-width and fit-page, and Ctrl+1 / Ctrl+2 / Ctrl+3 set actual size / fit width / fit page.

UNDER THE HOOD

```
ZoomMin 0.05, ZoomMax 5.0, ZoomStep 0.15; FitMode is None / Width / Page. See the deep-dive for the cursor-anchor math.
```

Undo `[Ctrl+Z]`

One history covering annotations, pages and whole-document edits.

IN THE GUI

Ctrl+Z reverses the last change; auto-repeat is ignored so one keypress is one undo.

UNDER THE HOOD

```
_undoStack holds UndoEntry records tagged by UndoKind - single or grouped annotation, clear-page, stamp batch, bookmark edit, page snapshot or a full document byte snapshot.
```

Command Line

Every core operation also runs headless from a terminal. No window, meaningful exit codes (0 success, 1 failed, 2 bad usage), and it works even while the app is open. Each command reuses the exact pipeline its GUI equivalent runs.

```
KillerPDF.exe --merge out.pdf a.pdf b.pdf scan.jpg
KillerPDF.exe --extract-pages in.pdf 1-3,5 out.pdf
KillerPDF.exe --split in.pdf pages\
KillerPDF.exe --decrypt locked.pdf open.pdf [--password p]
KillerPDF.exe --to-image in.pdf imgs\ --dpi 300 --format jpg
KillerPDF.exe --flatten in.pdf flat.pdf
KillerPDF.exe --print in.pdf --printer "HP LaserJet" --pages 1-4
KillerPDF.exe --ocr scan.pdf searchable.pdf --lang eng
KillerPDF.exe --batch-resave inDir\ outDir\ --log report.csv
KillerPDF.exe --help
```

Batch mode & the validation harness

--batch-resave resaves a single PDF or a whole folder tree headlessly through the standard open/save pipeline with per-file OK / SKIP / FAIL reporting. It exists so the standards-conformance harness (validation/, see Part III) can push thousands of corpus files through a real KillerPDF save and prove the output validates exactly as well as the input.

Rasterizing is rotation-safe at 150 or 300 DPI, printing drives the same 300 DPI flattened pipeline as the dialog, and --version / --help print and exit cleanly for scripting.

III

Reference

- The complete keyboard map and tool keys
- Six themes and the accent system
- Ten languages, standards conformance and the hard numbers

Keyboard Shortcuts

KillerPDF is fully keyboard-driven; global shortcuts pause only while you are typing in a text box or form field. Press F1 for the in-app overlay, which includes a visual keyboard view.

File

Key	Action
`Ctrl+O`	Open document
`Ctrl+N`	New blank document
`Ctrl+S / Ctrl+Shift+S`	Save in place / Save As
`Ctrl+P`	Print
`Ctrl+W`	Close tab
`Ctrl+Shift+W`	Close other tabs
`Ctrl+Q`	Close all tabs
`Ctrl+D / F4`	Document info

Tools

Key	Action
`V`	Select
`1 (or T)`	Text
`2 (or H)`	Highlight
`3 (or L or U)`	Line
`4`	Shapes
`5 (or D)`	Draw (ink)
`6 (or I)`	Image
`7 (or G)`	Signature
`8 (or C)`	Crop
`9 (or R)`	Rotate
`0 (or S)`	Stamp

Editing

Key	Action
`Ctrl+Z`	Undo
`Ctrl+Y / Ctrl+Shift+Z`	Redo
`Ctrl+C / Ctrl+V`	Copy / paste
`Ctrl+B / I / U`	Bold / italic / underline (editing a text box)
`Delete`	Delete selected annotation(s) or bookmark
`F2`	Rename the selected bookmark
`Enter / Escape`	Confirm / cancel
`Menu / Shift+F10`	Context menu at the selection

Help

Keyboard Shortcuts (continued)

Key	Action
F1 / Ctrl+?	Shortcuts overlay (list or keyboard view)
F12	About dialog

Navigation

Key	Action
Left / Right, PgUp / PgDn	Previous / next page (Two-Page: spread)
Home / End	First / last page
Up / Down	Scroll the view
Alt+Left / Alt+Right	Back / forward through the jump history
Ctrl+Scroll	Zoom around the cursor
Ctrl+= / Ctrl+-	Zoom in / out (Grid: step columns)
Ctrl+0	Reset zoom
Ctrl+1 / 2 / 3	Actual size / fit width / fit page
Middle drag / Space + drag	Pan / hand cursor
Ctrl+B	Collapse / restore sidebar
Ctrl+Shift+B	Move the sidebar to the other side
Ctrl+Tab / Ctrl+Shift+Tab	Next / previous tab

View

Key	Action
F5 / F6 / F7 / F8	Continuous / Single / Two-Page / Grid
F9 / wheel on the view button	Cycle through the four view modes
F10	Split pane: two documents side by side
F11 / Esc	Full screen
N	Invert document colors (night mode)
Shift+N	Also invert images (right-click the moon)
Ctrl+Shift+= / - / 0	App size: bigger / smaller / reset
Wheel on the title-bar logo	App size: bigger / smaller
Ctrl+Shift+1-6	Toolbar icon size and caption placement

OCR, Search & Select

Keyboard Shortcuts (continued)

Key	Action
`Ctrl+Shift+O`	OCR current page to clipboard
`Ctrl+Shift+I`	Begin OCR region
`Ctrl+F`	Toggle search
`F3 / Shift+F3`	Next / previous match
`Enter / Shift+Enter`	Next / previous match
`Ctrl+A`	Select all (annotations or text)
`Shift+Click`	Multi-select annotations

Tool Keys

The digits 1-9 and 0 mirror the toolbar positions left to right, and the original letters stay as aliases; tooltips append the key, e.g. `Highlight (2)`. Select is V alone - the Photoshop / Illustrator / Figma convention - which freed its digit so Shapes could take 4 and the numbers keep matching the toolbar. Esc with nothing left to cancel also drops back to Select, Acrobat-style.

Key	Action
`V`	Select
`T / 1`	Text
`H / 2`	Highlight
`L - U / 3`	Line
`4`	Shapes
`D / 5`	Draw (ink)
`I / 6`	Image
`G / 7`	Signature (opens picker)
`C / 8`	Crop
`R / 9`	Rotate (Transform window)
`S / 0`	Stamp

Themes & Accents

Six built-in themes, each a film-grain-textured palette. Three base themes also accept an accent overlay.

KillerPDF

Dark `#1EA54C`
The default - charcoal with brand green

KillerPDF

Light `#18843D`
Light surfaces, deep green accent

KillerPDF

Black `#00FF66`
High-contrast, neon-green chrome

KillerPDF

Blood `#F8C99E`
Deep oxblood with a cream accent

KillerPDF

Greed `#E0D49A`
Forest green with gold

KillerPDF

Cyanotic `#E0D49A`
Deep blue with a warm accent

Accent variants

The Dark, Light and Black themes each accept one of six accent overlays - this very page is themed with one.

Green	Red	Orange	Teal	Blue	Purple
`#1EA54C`	`#DD504B`	`#E8962C`	`#1FB8A8`	`#50AEE8`	`#B982E3`

Localization

KillerPDF ships ten UI languages. Strings are per-locale resource dictionaries keyed `Str_*`, with English always present as the base so any key a translation omits falls back automatically. Most UI binds the resources dynamically, so a language switch is instant.

Locale	Name
English (en-US)	Base locale, ~500 string keys
Spanish (es)	Espanol
German (de-DE)	Deutsch
French (fr-FR)	Francais
Turkish (tr-TR)	Turkce
Bengali (bn)	Bangla
Japanese (ja-JP)	Nihongo (new in 1.6.2)
Chinese Simplified (zh-CN)	Simplified
Chinese Traditional (zh-TW)	Traditional
Czech (cs-CZ)	Cestina (new in 1.6.5)

LocaleManager.Apply hot-swaps the merged dictionaries to theme + en-US + the chosen locale; code-built surfaces (context menu, toolbar captions, annotate bars, the shortcuts overlay) are rebuilt on a change. Contributions come in as pull requests against Strings/; every locale ships at full key parity with English.

File Handling & Robustness

KillerPDF is built to open the PDFs other viewers choke on, and to never lose your work to a malformed file.

Resilient open

- Network / UNC paths are copied locally first to avoid 9P short-read corruption; owner-restricted /Encrypt PDFs prompt for a password.
- An exception-driven fallback chain: xref corruption falls back to read-only then a PDFium repair; FlateDecode / EOF errors strip encryption and re-save losslessly; a zero-page page tree is guarded instead of crashing; anything else offers a repair.

Rotation-safe temp reload

SaveTempAndReload zeroes each page's /Rotate before saving (PDFium sizes from the unrotated MediaBox, so rotated content would clip) and restores it in memory; on xref failure it falls back to a PDFium save with FPDF_REMOVE_SECURITY or a PdfSharpCore import-repair.

Clean save & crash reporting

Saving writes a clean, annotation-free snapshot, burns stamps then annotations into a copy via PdfSharpCore XGraphics, and reopens the clean copy so future saves never double-burn. Every save also scrubs the hazards that corrupted files in older builds: dangling /Outlines references, degenerate crop boxes and dead signature values, and the cached PDFium link handle is released first. A 50-entry timestamped status ring buffer is dumped with structured crash logs to %LOCALAPPDATA%, auto-rotated at a 20 MB cap.

Standards Conformance

Every release is validated against veraPDF, the industry reference validator, across a 2,907-file public conformance corpus. The bar is zero regressions.

Every corpus file (the veraPDF test corpus for PDF/A-1/2/4 and PDF/UA-1/2, the Isartor PDF/A-1b suite and the TWG files) is validated pristine, resaved through the normal save pipeline via --batch-resave, and validated again. Any file that fails a rule after the resave that it did not fail before counts as a regression. A second pass runs qpdf --check on every original / resave pair.

Outcome (veraPDF 1.30.2)	Files
Corpus total	2,907
Resaved and revalidated	2,236
Refused (encrypted or unparseable; source untouched)	671
Conformance regressions	0 (one documented PDF 2.0 header limitation)
Files that came out more conformant	63
qpdf structural check worsened	0 (195 improved)

Patched at the source

Getting to zero meant fixing the write engine itself. PdfSharpCore is vendored under third_party/ with six patches, each marked in the source with the ISO clause it satisfies: no Producer/Creator stamping into an imported document's Info dictionary, no /ModDate rewrite at open (both break PDF/A's Info-to-XMP equivalence rule, ISO 19005-1 6.7.3), no transparency /Group injected onto every page (forbidden in PDF/A-1), stream /Length always matching the exact byte count, booleans written as the lowercase true / false keywords, and the debug verbose file layout removed.

The full report, the comparison script and reproduction instructions ship in the repo under validation/RESULTS.md; the summary lives on the technical page at KillerPDF.net.

Notable Engineering Details

A few engineering decisions worth calling out (the math behind the first two is in the deep-dive):

- **Zoom-stable coordinates.** Every annotation lives in the 2048-longest-side render-dim space, identical across all view modes, so marks never drift.
- **Decoupled bitmap resolution.** Pixels scale up to min(6144, 2048 * dpi * zoom) for sharpness while coordinates stay zoom-independent.
- **Continuous-view virtualization.** Only a window of pages around the viewport holds real bitmaps; released slots keep their exact height so scroll geometry never changes.
- **Per-tab LRU bitmap cache.** Frozen bitmaps keyed by (page, size-bucket, rotation), evicting whole tabs beyond the three most recent.
- **Reconcile-in-place tab strip.** A session-to-element cache diffs the strip instead of rebuilding it - no flicker on switch, drag or resize.
- **A vendored, patched write engine.** PdfSharpCore lives in the repo under third_party/ with six standards-conformance patches, proven by a 2,907-file veraPDF harness that runs on every release.
- **Two PdfSharp engines.** PdfSharpCore for the app, PDFsharp 6.2 (separate namespace) only for signing, behind a portable certificate-provider abstraction.
- **Self-extracting single exe.** Costura embeds managed DLLs; native Tesseract / Leptonica self-extract to a per-version cache and language packs download on demand.

Specifications & Constants

Selected constants and limits, drawn directly from the source.

Specification	Value
Render-dim longest side	2048 DIP (zoom-stable annotation space)
Bitmap resolution cap	6144 px = min(6144, 2048 * dpi * zoom)
Print / OCR render	300 DPI on demand / 2600 px (~300 DPI on Letter)
Image export DPI	24-1200 (default 150), PNG or JPEG
Zoom range / step	5% to 500%, 15% steps (0.05 / 5.0 / 0.15)
App-size scale range	70% to 250%, ~2% steps (accessibility, chrome only)
Render-cache tabs (LRU)	3 most-recently-used tabs
Per-file state memory	40 documents (DocStates)
Folder / zip import cap	50 files (MaxDropFiles)
Signature /Contents	16,384 bytes, SHA-256, whole chain
Default text annotation	Segoe UI, 14pt
Crash-log status ring	50 entries; logs rotate at 20 MB
Validation corpus	2,907 files (veraPDF + Isartor + TWG), zero-regression bar
UI languages / themes	10 locales / 6 themes plus accent variants

These are the real limits compiled into KillerPDF 1.7.1, taken straight from the source.

Glossary

Plain-language definitions for the terms used throughout this brochure.

Term	What it means
WPF	Windows Presentation Foundation, Microsoft's native desktop UI framework
partial class	A C# feature that lets one class be written across several files
PDFium	Google's open-source PDF rendering engine that turns a page into pixels
Docnet.Core	The .NET wrapper KillerPDF uses to drive PDFium
PdfPig	A .NET library that extracts text and word positions from a PDF
PdfSharpCore / PDFsharp	Libraries that write PDFs: saves and edits (vendored and patched), and signing
vendored	A library's source copied into the repo and built with the app, so it can be patched and audited
veraPDF	The industry reference validator for the PDF/A and PDF/UA standards
PDF/A	The ISO standard for archival PDFs; strict rules so files stay readable for decades
Tesseract	The open-source OCR engine that recognizes text in images
DIP	Device-independent pixel, a unit that stays the same apparent size on any display
render-dim	The internal page size (longest side 2048) where annotations are stored
xref	The cross-reference table, a PDF's index of where each object lives
MediaBox	The PDF entry that defines a page's full size
AcroForm	The PDF standard for interactive, fillable form fields
CMS / PKCS#7	The standard cryptographic container that holds a digital signature
OCR / DPI	Optical character recognition / dots per inch (300 DPI is print quality)

IV

Live Showcase

- A real interactive form to fill and save
- A printed-style form to mark up by hand
- Annotation pages seeded for screenshots

Interactive Feedback Form

Real AcroForm fields - open in KillerPDF (or any reader), fill them in, then Save.

Name

Role
IT / MSP technician

Email

Version used
1.7.1

Which tools do you use most? (tick all that apply)

- ☐ Annotation
- ☐ Forms
- ☐ OCR
- ☐ Signing
- ☐ Shapes
- ☐ Crop / pages
- ☐ Search
- ☐ Print

Overall rating

- ○ 1
- ○ 2
- ○ 3
- ○ 4
- ◉ 5 (best)
- ☐ Notify me of releases

Comments

Signature

Use the Signature tool (G) in KillerPDF to sign here.

Date

Sample Application Form

A printed-style form with no interactive fields - ideal for demoing fill-by-annotation, crop and stamp.

KILLERPDF - USER REGISTRATION & EVALUATION

Form KP-1 (Rev. 1.7.1) - Please print clearly in black ink

LAST NAME _____ FIRST NAME _____

ORGANIZATION _____ EMPLOYEE ID _____

EMAIL ADDRESS _____

PHONE _____ DATE (YYYY-MM-DD) _____ LICENSE No. _____

Edition requested: ☐ Portable (.exe) ☐ Installer ☐ Source build (GPL-3) ☐ Enterprise / MSP

Platform: ☐ Windows 10 ☐ Windows 11 ☐ Server 2019+ ☐ Other _____

Comments

SIGNATURE OF APPLICANT _____ DATE _____

LIVE SHOWCASE

Annotation Playground

Seeded content for screenshots. Reach for the toolbar (or the number keys) and mark this page up.

Highlight me (H / 2)

Drag the Highlight tool across this sentence and the mark flows along the words; Strikethrough and Underline give the other two styles.

Draw a shape (4)

Reach for the Shapes tool and drop a rectangle, ellipse or free-form polygon over this row, with an optional fill.

Draw here (D / 5)

Freehand ink area - round caps, adjustable width, optional eraser. The minimum-length guard ignores accidental taps.

Type a note (T / 1)

Click with the Text tool to drop a typewriter note, then style it from the floating bar.

Mark up a table

Tool	Shortcut	Output type
Highlight	H / 2	Highlight annotation
Shapes	4	Ink / highlight (filled)
Draw	D / 5	Ink annotation
Stamp	S / 0	Flattened overlay
Crop	C / 8	CropBox / TrimBox

LIVE SHOWCASE

Annotation Playground II

Seeded content for screenshots. Reach for the toolbar (or the number keys) and mark this page up.

Highlight me (H / 2)

Drag the Highlight tool across this sentence and the mark flows along the words; Strikethrough and Underline give the other two styles.

Draw a shape (4)

Reach for the Shapes tool and drop a rectangle, ellipse or free-form polygon over this row, with an optional fill.

Draw here (D / 5)

Freehand ink area - round caps, adjustable width, optional eraser. The minimum-length guard ignores accidental taps.

Type a note (T / 1)

Click with the Text tool to drop a typewriter note, then style it from the floating bar.

Mark up a table

Tool	Shortcut	Output type
Highlight	H / 2	Highlight annotation
Shapes	4	Ink / highlight (filled)
Draw	D / 5	Ink annotation
Stamp	S / 0	Flattened overlay
Crop	C / 8	CropBox / TrimBox

KillerPDF

Colophon

KillerPDF is designed and built by Steve the Killer.
This brochure was generated for KillerPDF v1.7.1; every figure and constant is drawn directly from the application source.

The wordmark is set in Typewriter A602, the app's logo face.
Body type is Helvetica; technical call-outs are set in Courier.

KillerPDF.net